

SourceTree

SourceTree 是一款免费的 Git 图形化客户端工具，提供了直观的图形界面来管理 Git 仓库，避免了在命令行中输入复杂的 Git 指令。用户可以轻松查看仓库的提交历史、分支结构、文件状态，并通过点击按钮完成代码的提交、推送、拉取、合并和分支管理等操作。它同时支持 Git 和 Mercurial 版本控制系统，并且可以关联 Bitbucket、GitHub 等远程仓库服务。

目录

SourceTree

- 目录

- Git

- 三个区域

- 仓库

- 节点

- 分支

- SourceTree使用

- 操作指令

- 一.仓库相关操作

- 创建本地仓库

- 打开本地仓库

- 克隆远程仓库

- 获取

- 关联远程仓库

- 二.节点相关操作

- 提交

- 重置

- 回滚提交

- 三.分支相关操作

- 新建分支

- 合并分支

- 删除分支

- 切换分支

- 制造冲突

- 解决冲突

- 检出

- 拉取

- 推送

- 四.其他

- 书写忽略文件

- 停止跟踪

- 创建补丁

- 应用补丁

- 变基

- 贮藏

- 丢弃

- 移除

- 创建标签

- 删除标签

- 五.常见

- 常见错误

Git

参考笔记[SourceTree使用教程](#)

Git 是一个内容寻址文件系统,核心部分是一个简单的键值对数据库 (key-value data store) , 属于分布式版本控制系统, 可以向 Git 仓库中插入任意类型的内容, 它会返回一个唯一的键, 通过该键可以在任意时刻再次取回该内容。

Git的底层文件**存储快照** (Snapshot) 而不是**差异** (Delta)

Git的三个核心对象: **Blob文件**、**Tree目录树**、**Commit提交**

Blob (数据对象 / 文件快照)

- **作用**: 它是 Git 用来**存储文件具体内容**的核心单元。但它**只存文件内容, 不存文件名**。

Tree (树对象 / 目录结构)

- **作用**: 它相当于操作系统的**文件夹**, 用来组织管理文件结构。
- 一个 Tree 对象会记录当前目录下有哪些文件 (即 Blob 对象) 和子文件夹 (即其他 Tree 对象), 并记录它们的**文件名、文件权限和对应的 SHA-1 哈希值**。

Commit (提交对象 / 历史快照)

- **作用**: 它是对一次提交操作的**完整封装**。
- 一个 Commit 对象会记录: 当前根目录的 **Tree 对象哈希** (代表这一刻的所有文件状态)、**父 Commit 的哈希** (指向上一个版本, 形成链条)、作者信息、提交时间以及**提交说明 (Log)**。

三个区域

Git 将本地仓库划分为三个工作区域: 工作区 (Working Directory)、暂存区 (Staging Area / Index) 和版本库 (Repository)。

- **工作区**是项目在磁盘上的实际目录, 即编辑代码、修改文件的地方, 所有未跟踪或已修改但未暂存的文件都存在于工作区中。
- **暂存区**是一个临时的存储区域, 用于存放准备提交的文件变更, 当执行 `git add` 时, 文件的快照被保存到暂存区, 等待下一次提交。
- **版本库**则是存储所有已提交历史记录数据库, 当执行 `git commit` 时, 暂存区中的内容被永久保存到版本库中, 形成一个不可变的历史节点。

三个区域的标准工作流是: 在工作区中修改文件 → 通过 `git add` 将变更添加到暂存区 → 通过 `git commit` 将暂存区内容提交到版本库。

仓库

仓库 (Repository) 是 Git 进行版本控制的核心单元，**本质上是一个存储所有文件和文件变更历史的数据库。**

一个仓库包含了项目从创建到当前时刻的所有版本信息，包括每次提交的快照、提交说明、作者信息、时间戳以及分支结构等元数据。

在物理层面，本地仓库表现为项目根目录下的 `.git` 隐藏文件夹，该文件夹记录了所有版本数据，而包含 `.git` 文件夹的整个目录称为工作区。

仓库分为**本地仓库**和**远程仓库**两种形式：

本地仓库位于开发者自己的计算机上，包含完整的版本历史记录，支持离线操作；

远程仓库（如 GitHub、GitLab、Bitbucket 等平台上托管的仓库）则用于团队协作和代码备份，开发者通过 `push` 将本地提交同步到远程仓库，通过 `pull` 或 `fetch` 从远程仓库获取他人的变更。

节点

节点 (Node / Commit) 在 Git 中指的是**每一次提交所生成的唯一历史记录**，每个节点都有一个唯一的哈希值（如 `a1b2c3d4e5f6...`）作为标识符。

每个提交节点 (Commit) 的哈希值 (SHA-1 算法) 是 **40 位**，40 个十六进制字符，但在 SourceTree 或命令行中常显示**前 7 位**

每次提交都会生成一个新的节点，该节点包含了当时暂存区的完整文件快照、提交说明、作者信息、时间戳以及指向上一个节点（父节点）的指针，从而串联形成一条不可逆的提交链。

在提交历史中，除了初始提交（根节点）外，每个节点都至少有一个父节点，合并提交则有两个或多个父节点。当前分支最新的一次提交对应的节点称为 HEAD，表示当前工作区所处的版本位置。

分支

分支 (Branch) 是 Git 中实现并行开发和版本隔离的核心机制，**本质上是一个指向特定提交节点的可变指针。**

每次创建新分支时，实际上只是创建了一个新的指针，指向当前所在的提交节点，而不会复制任何文件内容，因此分支创建几乎是瞬时完成的。

当在分支上进行新的提交时，该分支指针会自动向前移动到最新的提交节点，而其他分支的指针保持不变，从而实现了代码的隔离和并行开发。

分支的典型用法包括：`master`（或 `main`）作为主分支用于发布稳定版本，`develop` 作为开发分支用于集成新功能，`feature/*` 作为功能分支用于开发特定功能，`hotfix/*` 作为热修复分支用于紧急修复线上问题。在 Git 中切换分支时，工作区的文件会被替换为对应分支指向的快照版本，从而实现分支间的上下文切换。与集中式版本控制系统不同，Git 的分支是轻量级的，创建、切换和合并分支的成本极低，因此鼓励频繁创建和使用分支。

SourceTree使用

操作指令

一.仓库相关操作

创建本地仓库

点击 **New** → **Create Local Repository**，选择存放路径并设置仓库名称，点击 **Create** 即可完成创建。

```
git init
```



Create a repository

目标路径:

名字:

Git

☐ 在账户中创建仓库:

打开本地仓库

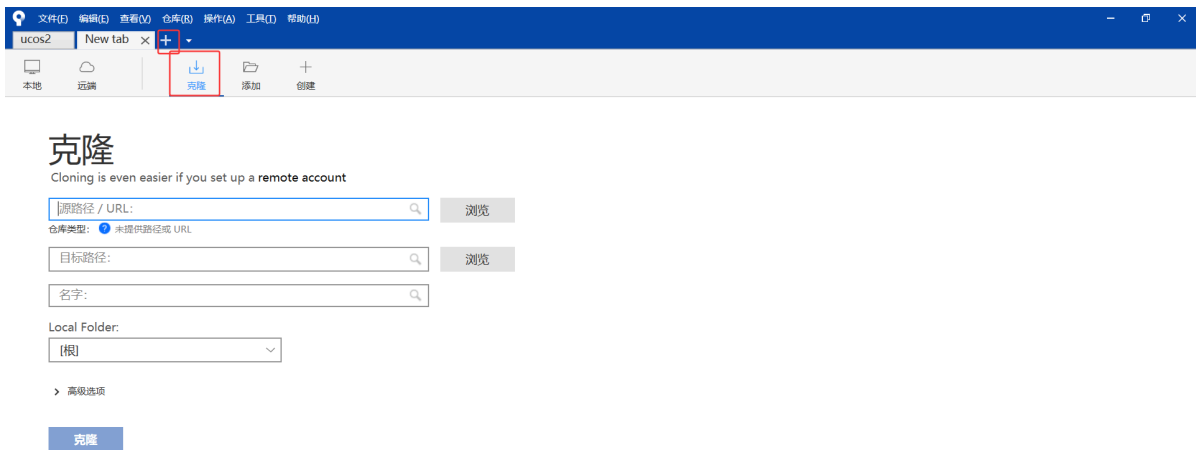
点击 **Open** 按钮，选择本地包含 **.git** 文件夹的项目目录，即可打开并添加到仓库列表。

无需指令，直接进入目录即可

克隆远程仓库

点击 **Clone** 按钮，填写远程仓库 URL 和本地存放路径，点击 **Clone** 即可将远程仓库完整下载到本地。

```
git clone <远程仓库地址>
```

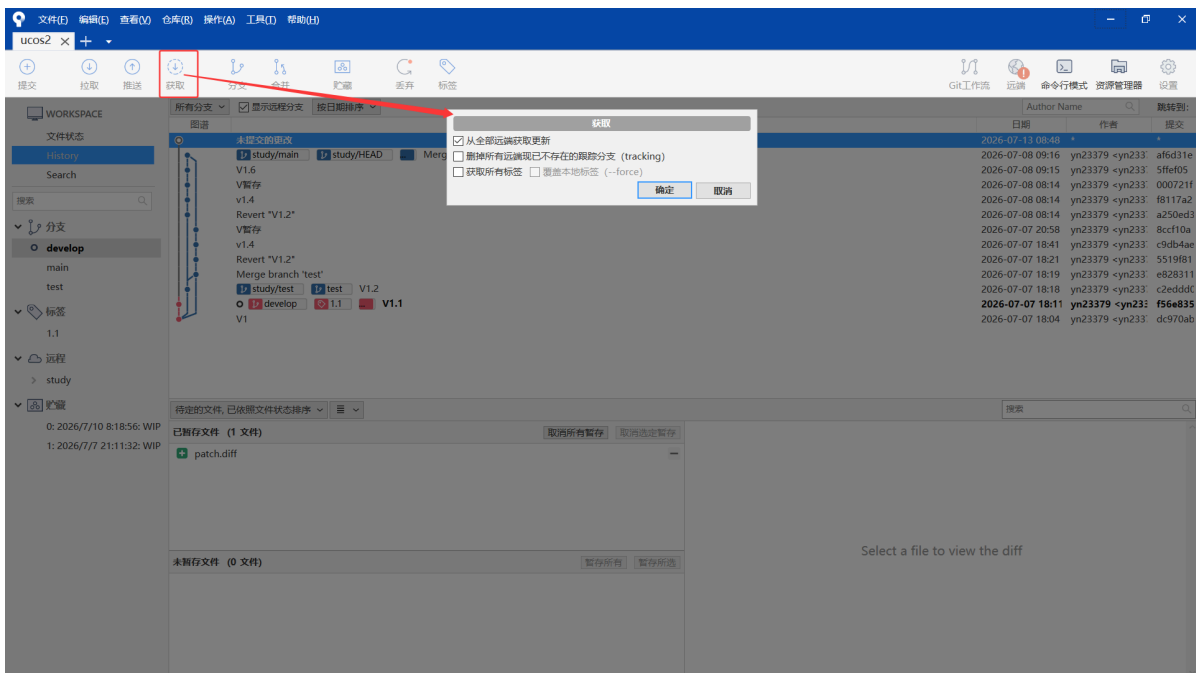


获取

点击 **Fetch** 按钮，从远程仓库拉取最新提交记录到本地，但**不会自动合并**到当前分支。

获取**只下载**远程仓库的最新提交历史，不能在本地仓库看到代码

```
git fetch origin
```



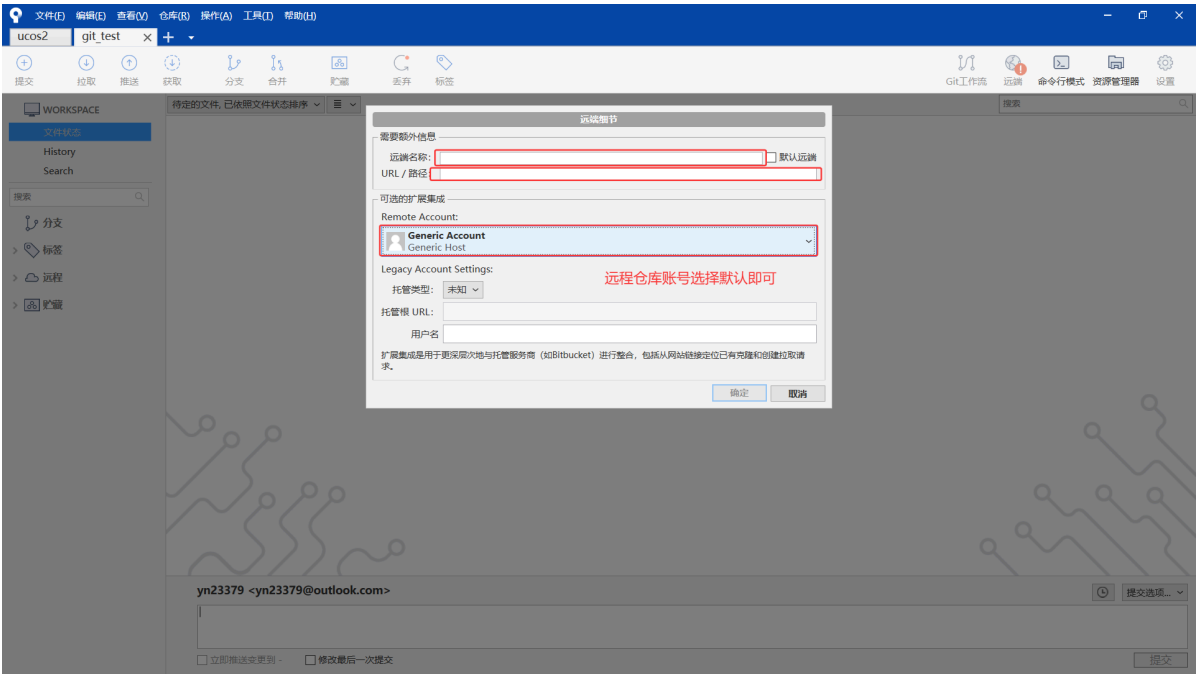
获取Fetch和拉取Pull的区别：

获取是只下载远程仓库的最新内容，不会修改本地代码，本地工作区不改变，不会自动合并到当前工作分支。

拉取是 `git fetch + git merge` 的组合，会修改本地代码，如果有未提交的改动，可能被阻止或产生冲突

关联远程仓库

点击 仓库 ——> 仓库设置 ——> 远程仓库 的设置里选择 添加，在 远端名称 填入 `origin`，URL 填写远程仓库地址，点击 OK 保存。

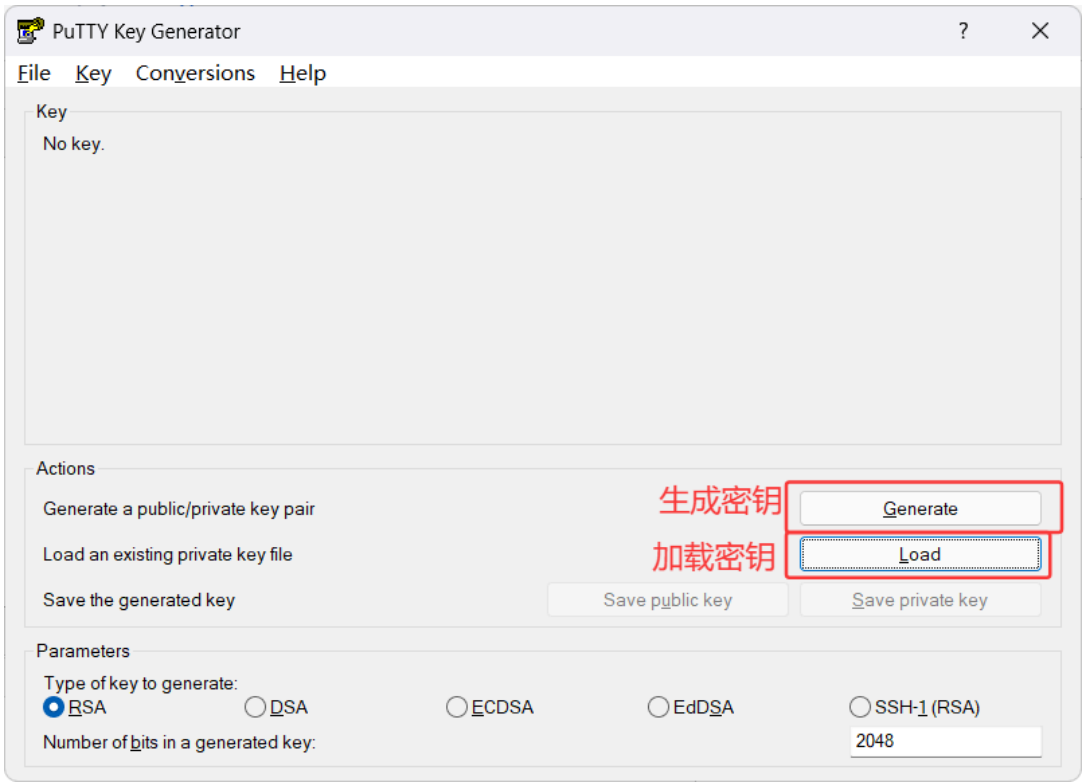


```
git remote add origin <远程仓库地址>
```

HTTPS 与 SSH 的区别

- **HTTPS**：走网页授权，使用账号密码/Token，无需额外配置。
- **SSH**：走加密密钥验证，配置略微复杂，需绑定公钥。

ssh密钥配置：



1. 在 SourceTree 顶部菜单栏，点击 工具 (Tools) → 创建或者导入ssh密钥。

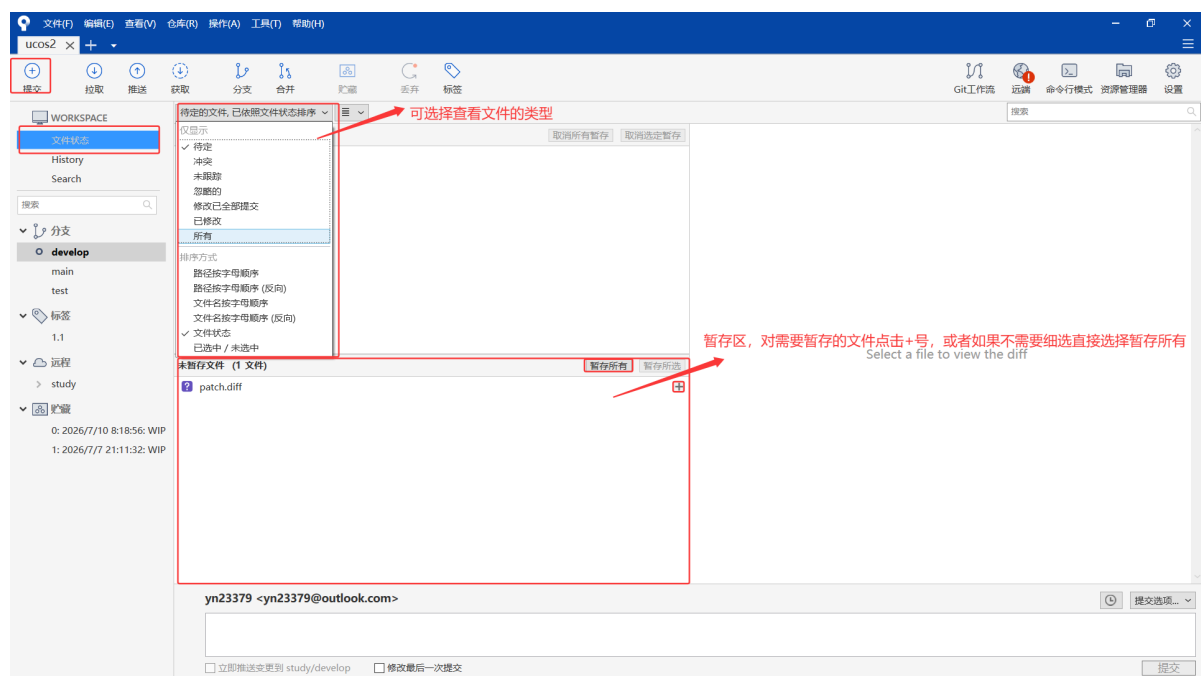
- 在弹出的路径选择框中，找到你存放 GitHub SSH 密钥的文件夹，选中 `id_rsa` 这个文件（注意不要选 `id_rsa.pub`，要选没有后缀的那个私钥文件）。
- 点击打开，然后点击底部的 `确定`。

二.节点相关操作

提交

在提交面板中填写提交说明，勾选需要提交的文件，点击 `Commit` 完成提交。勾选 `Push commits immediately` 可在提交后自动推送。

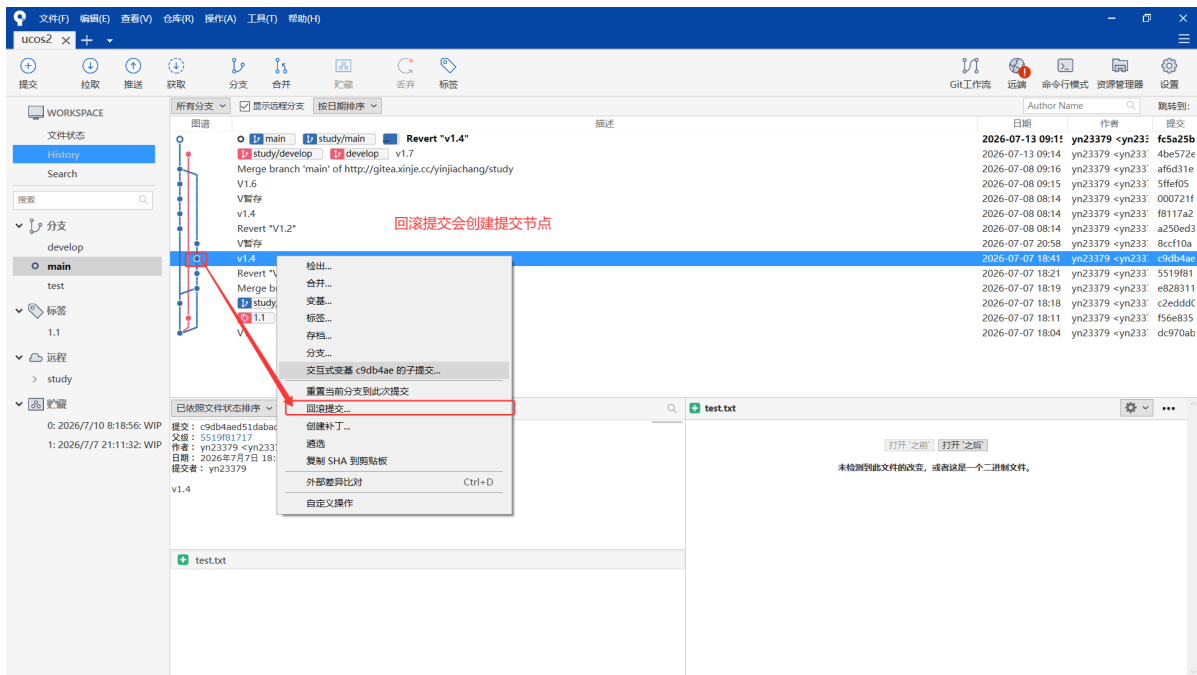
```
git add <文件名>      # 添加指定文件到暂存区
git add .              # 添加所有变更到暂存区
git commit -m "提交说明" # 提交到本地仓库
```



重置

右键点击历史中的目标节点，选择 `重置当前分支到此次提交`。

重置就是将当前分支的指针（HEAD）强制移动到指定的某个历史节点上。



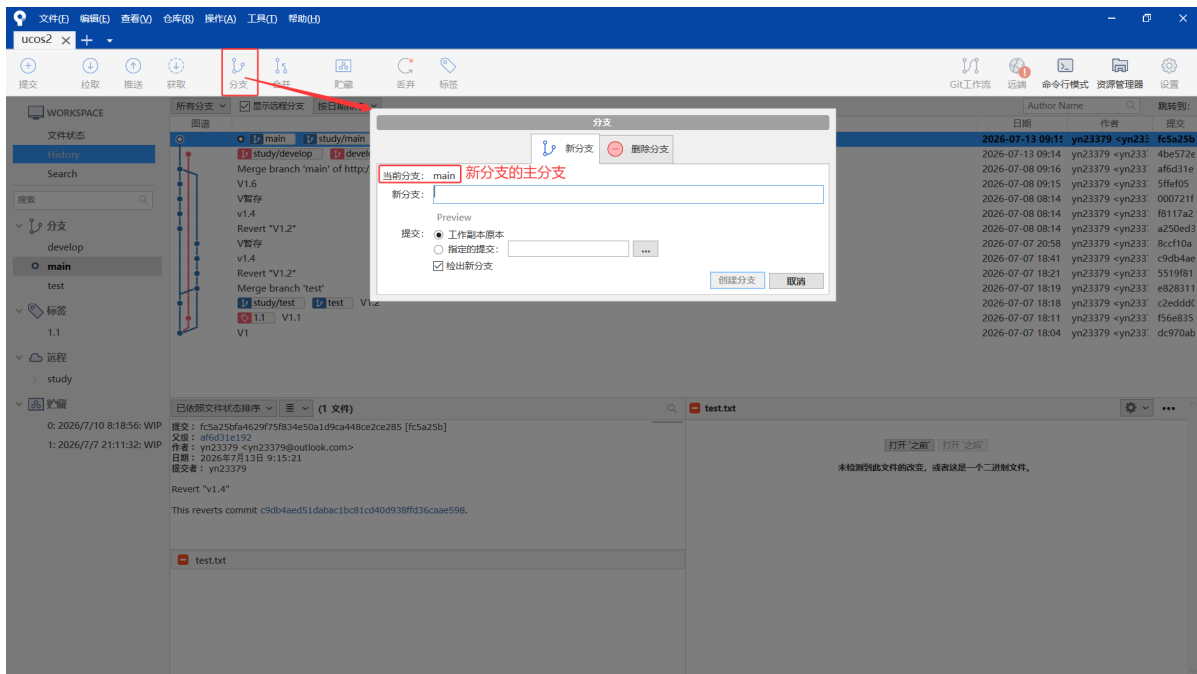
回滚算的是从目标节点到HEAD节点的差值并反向打补丁，重置算的是目标节点到当前节点的差值并决定是保留 (--soft/--mixed) 还是丢弃 (--hard)。

三.分支相关操作

新建分支

点击 **Branch** 按钮，在弹出的对话框中输入分支名称，点击 **Create Branch** 即可基于当前提交创建新分支。

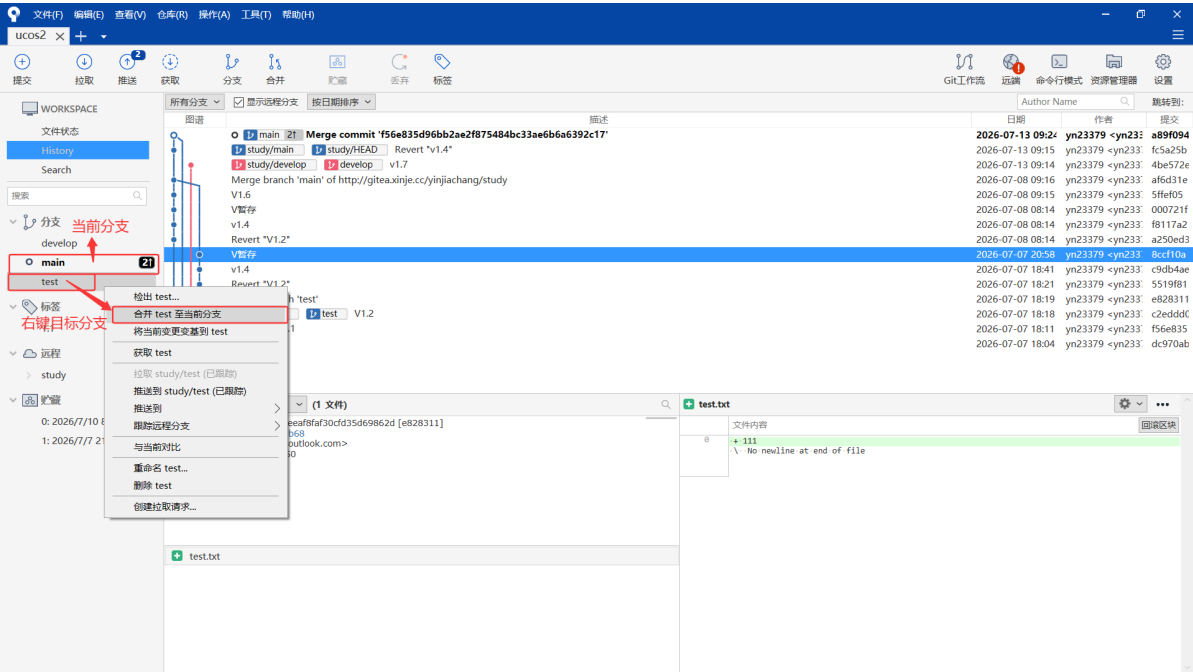
```
git branch <分支名>          # 创建分支
git checkout -b <分支名>      # 创建并切换到新分支
```



合并分支

切换到目标分支（如 `master` ），右键点击要合并的分支，选择 `Merge` 即可将指定分支合并到当前分支。或者在分支栏右键目标分支，选择合并至当前分支

```
git merge <分支名>
```



在合并代码之前都需要将需要合并的分支拉取到最新状态，避免覆盖别人的代码，或者丢失一些重要文件

在 SourceTree 里，右键点击某个对象，选择“合并...”：

右键对象	菜单选项	实际执行的操作
另一个分支（如 <code>feature</code> ）	合并 <code>feature</code> 到当前分支	<code>git merge feature</code>
某个历史节点（提交哈希）	合并这个提交	<code>git merge <commit-hash></code>
远程跟踪分支（如 <code>origin/main</code> ）	合并 <code>origin/main</code> 到当前分支	<code>git merge origin/main</code>

本质：无论选的是分支名还是历史节点，Git 最终都是把那个节点指向的内容合并到当前分支。

合并不一定产生新节点，如果当前分支没有动过，只是指针后移（快进），就不产生新节点；但凡当前分支也有自己的新提交，合并时一定会产生一个合并提交。

删除分支

右键点击要删除的分支，选择 `Delete` 即可删除本地分支。如需删除远程分支，在 `Remotes` 下右键选择 `Delete Remote Branch`。

```
git branch -d <分支名>           # 删除已合并的本地分支
git branch -D <分支名>           # 强制删除本地分支
git push origin --delete <分支名> # 删除远程分支
```

远程分支的删除不受本地是否已合并的限制。但 Git 的完整版本历史只保存在本地仓库——远程仓库不包含其他开发者本地的提交。

对比项	普通删除（ <code>git branch -d</code> ）	强制删除（ <code>git branch -D</code> ）
安全提示	有保护机制。如果该分支上的提交还没有合并到当前所在的分支（或其他分支），Git 会直接拒绝删除并报错。	无保护机制，直接暴力删除。即使该分支上的提交完全没有被合并过，Git 也会直接把它删掉。
触发警告	报错提示： <code>error: The branch '分支名' is not fully merged.</code>	没有任何警告，直接成功。
数据恢复	因为没被删掉，可以通过合并或切换到该分支继续工作。	被强制删掉的、没有合并过的提交，在 Git 里会变成“孤儿提交”。虽然能通过 <code>git reflog</code> 追回，但非常麻烦且容易永久丢失。
适用场景	标准安全做法。用于删除已经完全合并进主分支的功能分支。	必须删的情况：1. 开发到一半写废了，不想合并也不想要的废弃分支。2. 分支名写错了，想删了重开。

切换分支

在分支列表中双击目标分支名称，即可切换工作区到该分支。也就是把 `HEAD` 指针指向另一个分支，然后把工作区的文件替换成那个分支的最新内容。

```
git checkout <分支名>
```

切换分支时工作区文件会被替换为目标分支指向的快照版本。

切换分支前，务必确保工作区处于“干净”状态，即所有未提交的改动都已通过提交（Commit）保存为历史节点，或通过贮藏（Stash）临时封存。

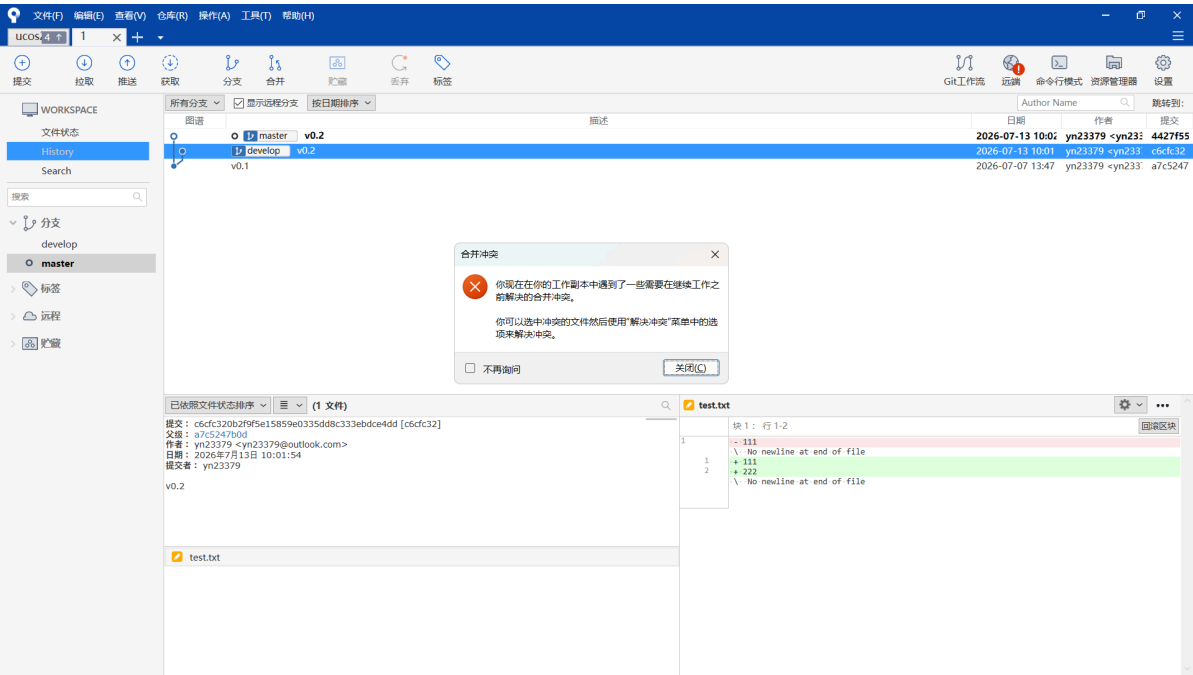
未提交的文件是绑定在当前工作区和暂存区的，不跟分支走。

切换分支时，未提交的改动（未追踪/已追踪）都会被动“隐藏”起来，直到你切回原来的分支才会重现。想带它走，必须先 Commit 或者 Stash。

若确认改动无用，可直接丢弃（Discard）。若工作区存在未提交改动且与目标分支的文件修改发生冲突，Git 将自动阻止切换，以防代码意外丢失。这是 Git 的安全保护机制，而非错误。

制造冲突

在两个不同的分支上修改同一文件的同一区域，然后尝试将后修改的分支合并到先修改的分支，此时 Git 无法自动合并，产生冲突。



如图，在develop分支下对text文档的第一行做修改并提交，切换到main分支也对text文档的第一行做修改并提交，在合并develop分支到main分支时就产生了冲突。

text文件内容被修改为：

冲突文件中的 <<<<<<<、=====、>>>>>>> 标记分别标识当前分支和合并分支的代码。

- <<<<<<< HEAD 到 ===== 之间：是当前所在的分支（也就是执行 git merge 时，正在工作在这个分支上）保留下来的代码。
- ===== 到 >>>>>>> develop 之间：是试图合并进来的那个分支**（这里是指 develop 分支）带过来的代码。

```
111
<<<<<<< HEAD
333
=====
222
>>>>>>> develop
```

也就是

```
111          <-- 冲突之外的正常代码
<<<<<<< HEAD          <-- 冲突区域开始（标记当前所在分支）
333          <-- 暂存区/当前分支（HEAD）的代码
=====          <-- 分界线
222          <-- 合并过来的分支（develop）的代码
>>>>>>> develop      <-- 冲突区域结束（标记合并过来的分支）
```

冲突产生的根本原因：当 Git 尝试合并两个分支（或两个提交）时，如果这两个分支在**同一文件的同一位置**做出了**不同的修改**，Git 无法自动判断应该保留哪一个版本，就会暂停合并过程并向用户报告冲突。

触发冲突的具体操作包括： `git merge`（合并分支）、`git pull`（拉取远程代码并合并）、`git rebase`（变基）、`git cherry-pick`（遴选特定提交）等所有涉及“将两个不同历史节点的代码改动合并到一起”的操作。

只要两个改动在文件内容上没有“重叠冲突区域”，Git 就能自动完成合并；一旦发生重叠且内容不同，便必然产生冲突，必须由开发者手动裁决最终代码。

解决冲突

冲突出现时，SourceTree 会列出冲突文件。在外部编辑器中打开冲突文件，手动编辑保留最终需要的代码，保存后回到 SourceTree，勾选已解决的文件，点击 `Commit` 提交合并结果。

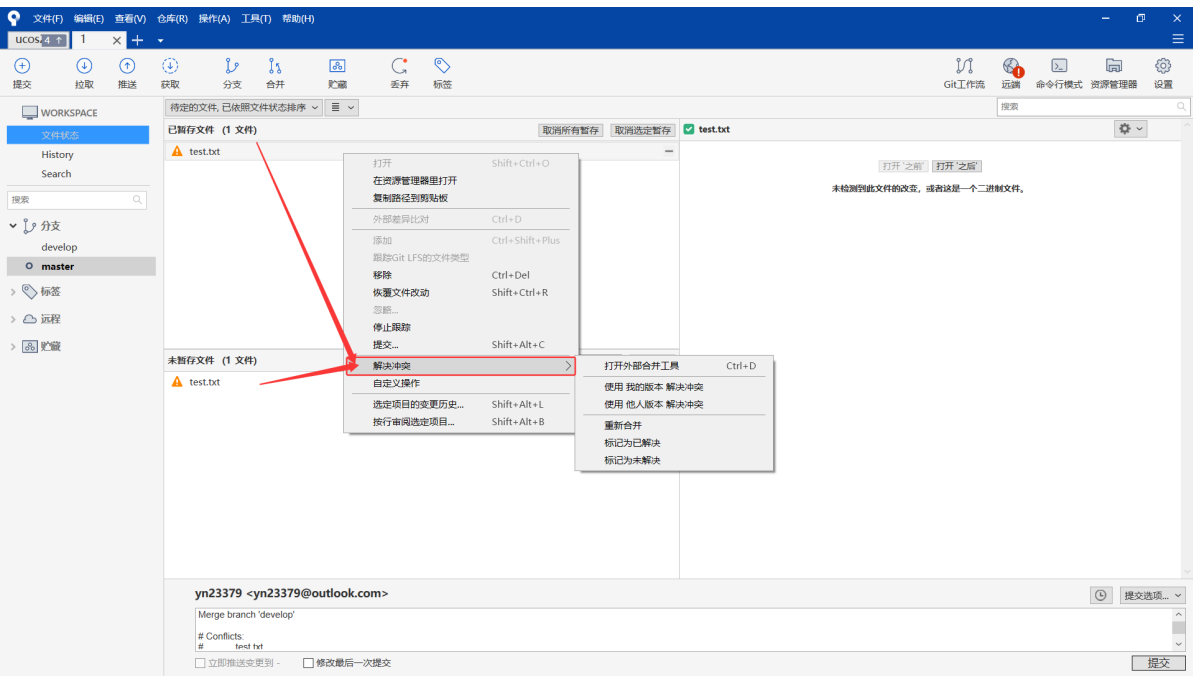
```
# 手动编辑冲突文件，删除 <<<<<<<、=====、>>>>>>> 标记
git add <冲突文件>
git commit -m "合并并解决冲突"
```

方式	操作路径	本质	适用场景
1. 手动编辑	打开冲突文件 → 删除 <<<<<<<、=====、 >>>>>>> 标记 → 保留最终代码	直接修改文件内容	需要精细控制合并结果，或两种改动都需要保留并重组时
2. 使用外部合并工具	右键冲突文件 → “解决冲突” → “打开外部合并工具”	调用已配置的第三方对比/合并工具（如 Beyond Compare、KDiff3、Meld）进行可视化合并	需要图形化对比左右版本，直观选择保留哪一侧的改动
3. 选择“使用我的版本”	右键冲突文件 → “解决冲突” → “使用我的版本”	直接丢弃目标分支的改动，完全保留当前分支的版本	明确当前分支的改动是正确的，目标分支的改动不需要
4. 选择“使用他们的版本”	右键冲突文件 → “解决冲突” → “使用他们的版本”	直接丢弃当前分支的改动，完全采用目标分支的版本	明确目标分支的改动是正确的，当前分支的改动不需要

方式	操作路径	本质	适用场景
5. 全部解决并标记	菜单栏 → “操作” → “解决冲突” → “标记为已解决”	手动编辑完成后，或选择“使用我的/他们的版本”后，标记该文件冲突已解决	已在外部工具或手动编辑中完成冲突处理，需要告诉 Git 该文件已准备就绪可提交

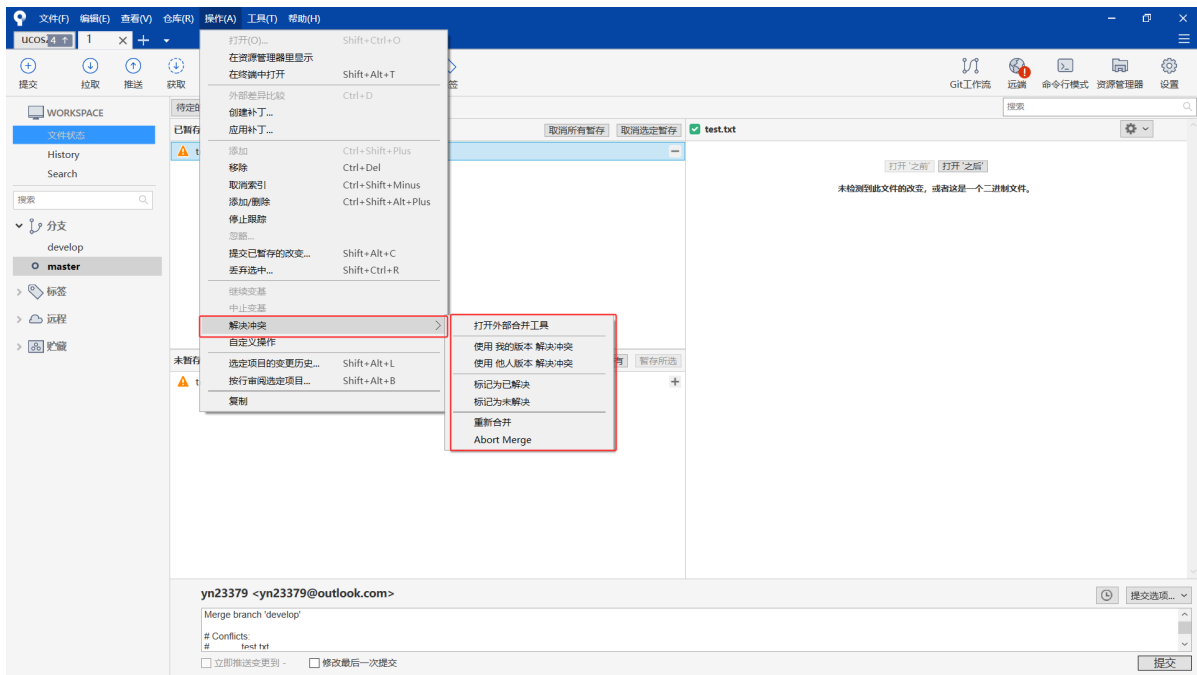
• 右键单击冲突文件 → “解决冲突” 子菜单：

- 打开外部合并工具
- 使用我的版本
- 使用他们的版本
- 标记为已解决



• 顶部菜单栏 → “操作” → “解决冲突”：

- 打开外部合并工具
- 使用我的版本
- 使用他们的版本
- 标记为已解决
- （部分版本还支持“全部标记为已解决”）



冲突文件右键即可选择四种核心处理方式：手动编辑、外部工具对比合并、直接选用“我的版本”、直接选用“他们的版本”。完成任何方式后，必须“标记为已解决”才能继续提交。

检出

检出（Checkout）主要用于切换分支或恢复工作区文件。

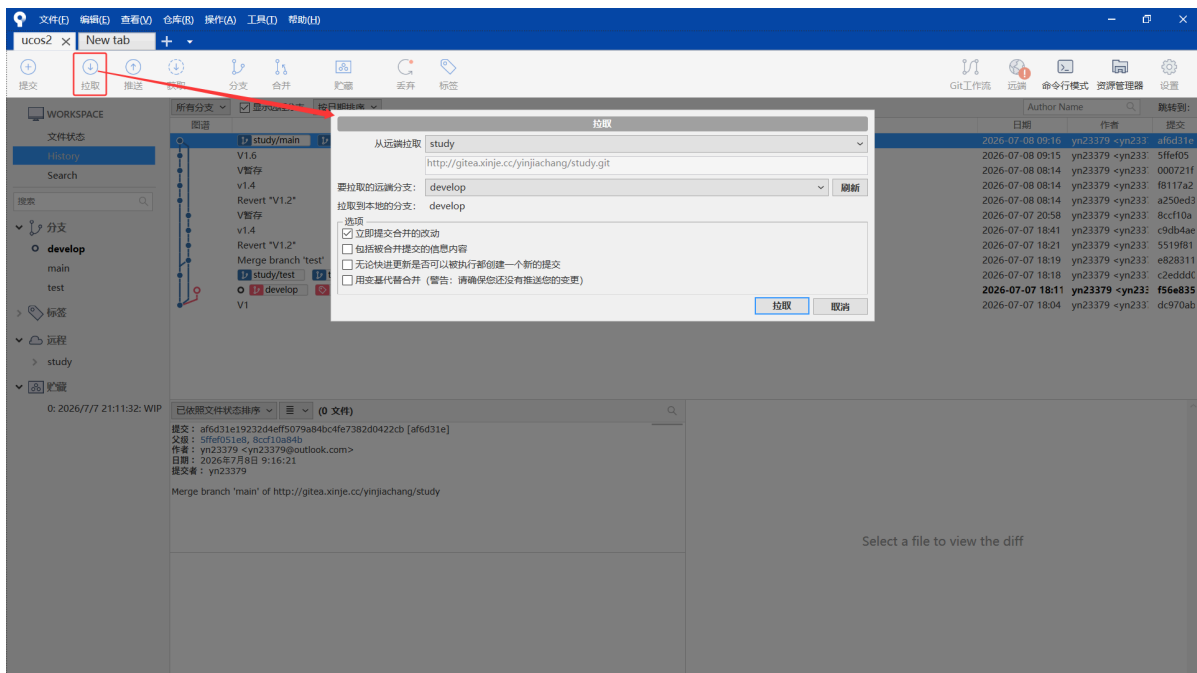
- 检出远程分支的本质是：拉取远程分支到本地，并绑定上下游关系。

拉取

点击 Pull 按钮，从远程仓库拉取最新代码并自动合并到当前分支。

`git pull` = `git fetch`（获取远程更新） + `git merge`（合并到本地分支）

```
git pull origin <分支名>
```



拉取会尝试把本地的提交历史和远程的提交历史拼接到一起。

执行拉取之前，也必须保证**本地的暂存区（Stage）是干净的**。如果本地有**未提交的修改**，即未暂存或已暂存但没 commit，此时 `git pull` 尝试合并，Git 会报错并**直接拒绝合并**，或者造成文件状态混乱，导致 SourceTree 里按钮变灰。

推送

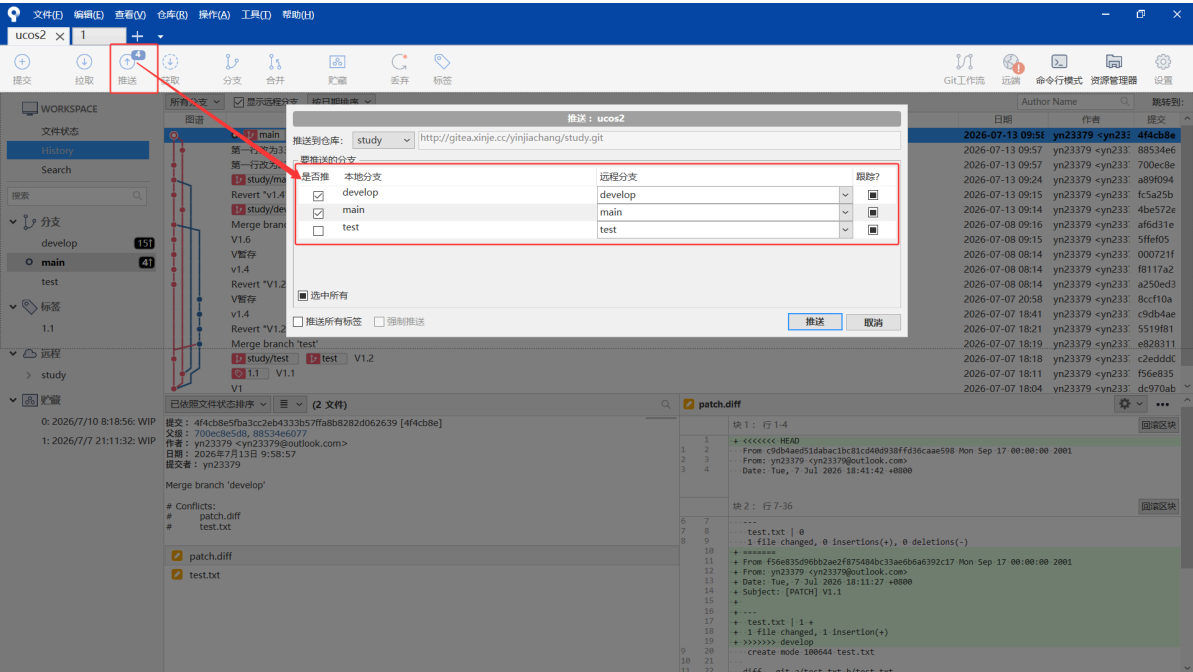
点击 `Push` 按钮，将本地提交推送到远程仓库。

```
git push origin <分支名>
```

推送前，需要先拉取（Pull）

如果是合作的分支，绝不能强制推送（Push -f）

如果推不上去，定是远程有更新，拉下来合并完再推。



四.其他

书写忽略文件

在仓库根目录创建 `.gitignore` 文件，每行写入一个忽略规则（如 `*.tmp`、`build/`），Git 将不再跟踪这些文件。

```
# 无需指令，直接编辑 .gitignore 文件
```

`.gitignore` 只对**未被跟踪（Untracked）**的文件生效。已经被 Git 跟踪的文件，即使加入 `.gitignore`，Git 仍然会继续跟踪其变化，不会立刻停止跟踪。

编译中间文件（.o/.obj/.lib/.a等），IDE/编辑器配置文件（个人配置不同），临时文件和日志文件，敏感信息和密钥文件等适合放入 `.gitignore` 文件。

格式：

- **逐行匹配：**每行一个忽略规则，`#` 开头表示注释。
- **通配符：**
 - `*` 匹配任意字符（如 `*.log`）。
 - `?` 匹配单个字符。
 - `**` 匹配多级目录（如 `build/**/temp`）。
- **路径定位：**
 - `/` 开头：仅匹配根目录（如 `/target`），不匹配子目录。
 - `/` 结尾：仅匹配目录（如 `build/`）。
- **否定规则：**`!` 开头表示“不忽略”，覆盖前面的规则（如 `*.txt` 后加 `!important.txt`）。

```
# 1. 注释：忽略所有日志文件
*.log
*.tmp

# 2. 忽略编译生成的文件夹（/ 开头匹配根目录下的 build，子目录的不影响）
/build/
/out/

# 3. 忽略依赖包目录（** 匹配任意层级的目录）
node_modules/
**/vendor/

# 4. 忽略本地环境配置和系统文件（含密码/密钥）
.env
.DS_Store
.idea/
.vscode/

# 5. 否定规则（例外）：虽然忽略了所有 *.txt，但这个文件需要保留
*.txt
!important.txt

# 6. 忽略某个特定的文件，但在特定目录下保留
doc/readme.md
!src/doc/readme.md
```

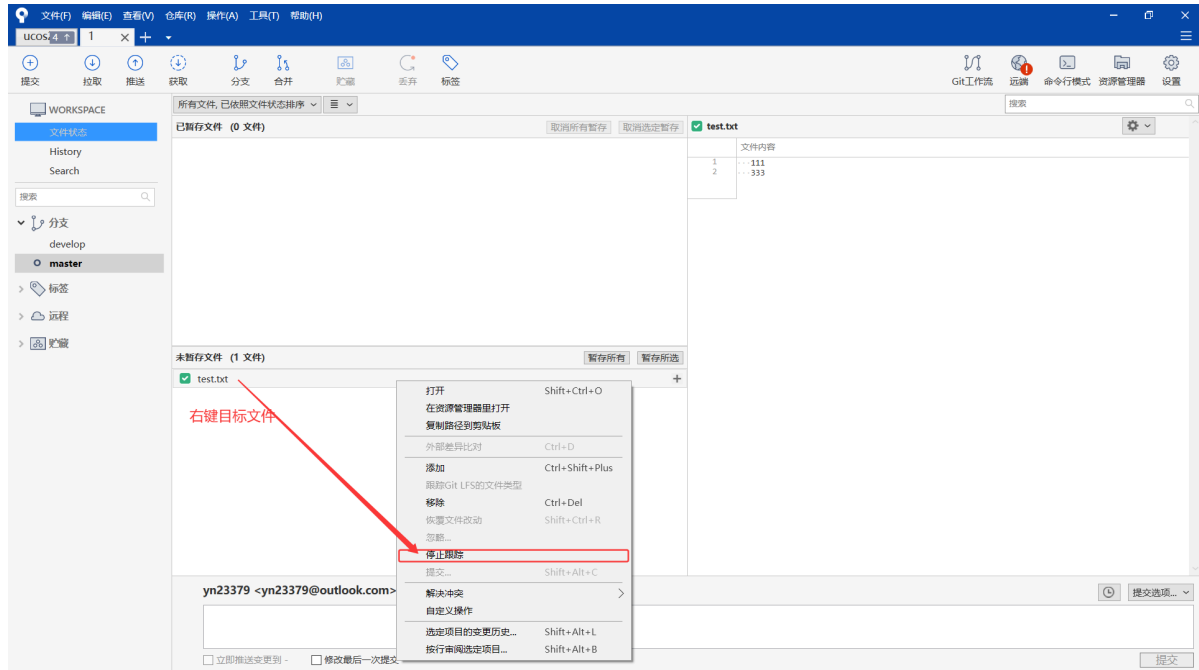
停止跟踪

如需停止跟踪已版本控制的文件，在文件状态，未暂存文件里右键点击文件选择 **Stop Tracking**，文件会从版本库中移除但保留在工作区。未被跟踪的文件Git会忽视它的修改，也可以让其重新被跟踪

```
git rm --cached <文件名>
```

停止跟踪是为了避免信息泄露，减少仓库体积，减少不同环境配置的冲突。**也就是让 Git 不再监控某个文件的变化，把它从版本库的记录里剔除出去。**

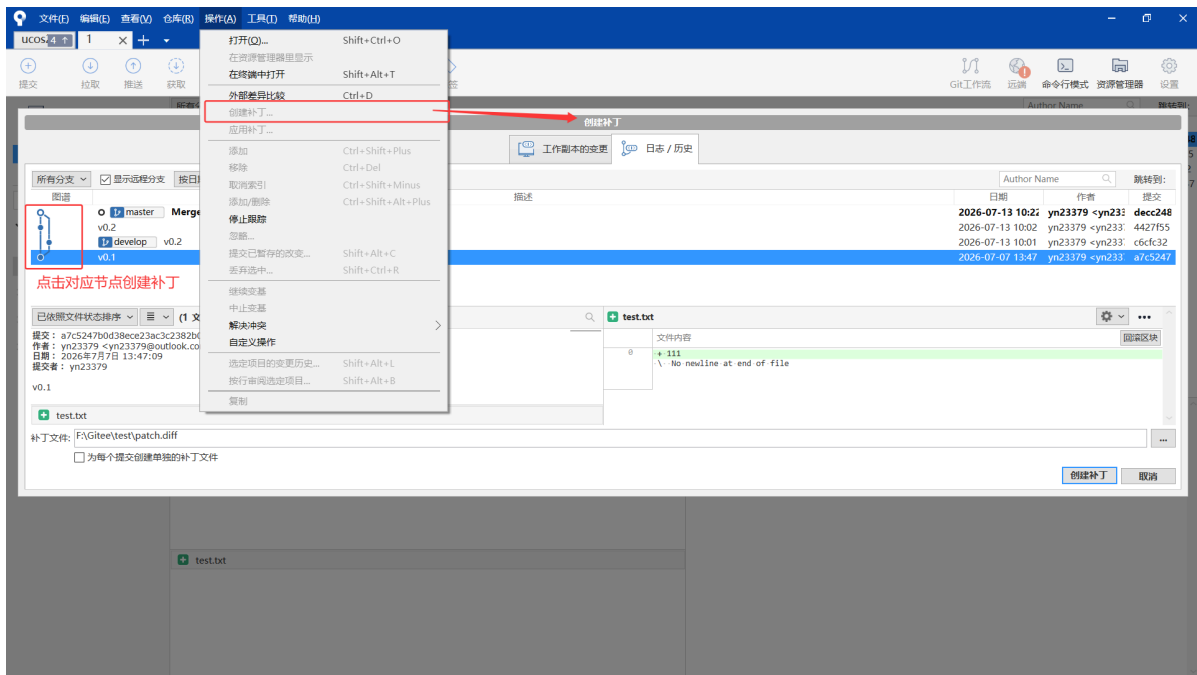
对文件进行停止跟踪就是把它从 **Git 的追踪列表中移除**。它会将文件从暂存区（Index）中删除，但保留硬盘上的物理文件。



创建补丁

1. 右键点击某个提交节点，选择 **创建补丁** 即可生成该提交的补丁文件（.patch）。

```
git format-patch -1 <节点哈希>
```

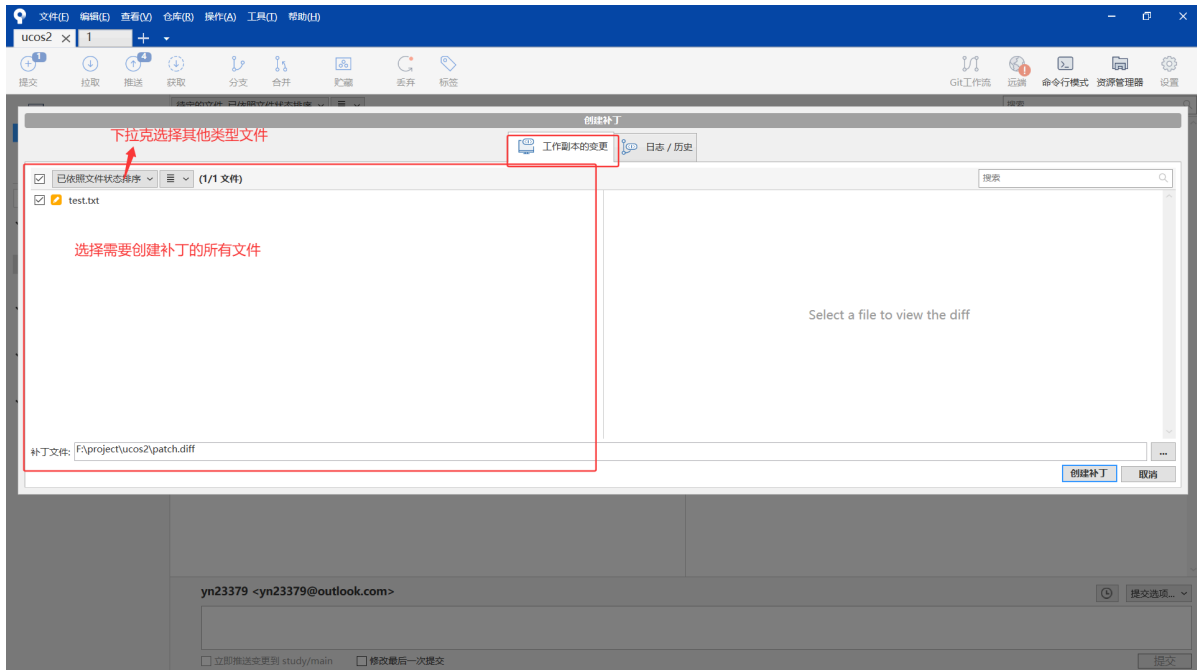


2. 在提交历史中按住 `Ctrl` 选中多个连续或不连续的提交节点，右键选择 `Create Patch`，SourceTree 会为每个选中的提交分别生成独立的补丁文件。

```
git format-patch <起始哈希>..<结束哈希>
```

3. 在操作下拉栏选择创建补丁，对暂存区里还未提交的文件进行补丁的创建。

```
git diff HEAD > patch.diff
```

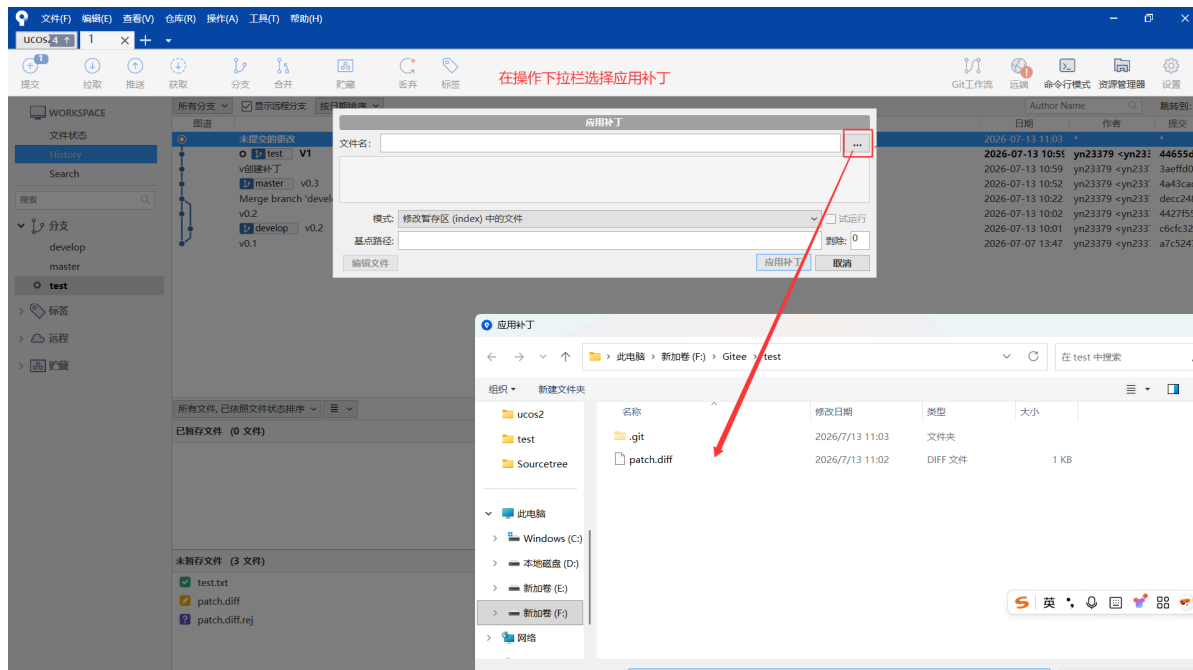


应用补丁

1. 右键点击仓库，选择 `Apply Patch`（版本差异的不同可能操作也不同，顶部的操作栏里有些版本也有创建和应用补丁），选择补丁文件即可将补丁应用到当前仓库。

应用补丁之后

```
git apply <补丁文件.patch>
```



2. 使用 `git am` 应用由 `git format-patch` 生成的 `.patch` 文件，应用后会产生新的提交节点，保留原提交的提交说明和作者信息。

```
git am <补丁文件.patch>
```

应用补丁只修改工作区的文件，不会创建任何提交节点。

3. 也可以使用Beyond Compare、KDiff3 或 WinMerge这些第三方工具使用。

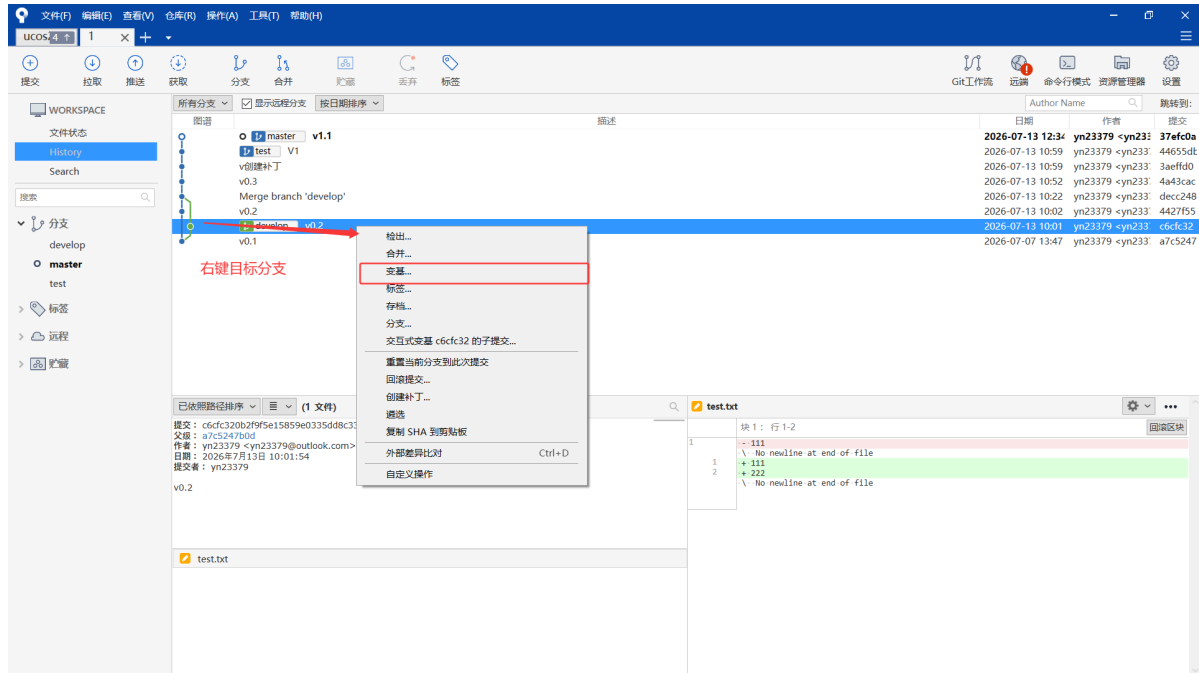
4. 补丁本质上是“代码片段”，可以当做粘贴板来用。

可以在要插入这些修改的前一个提交节点里右键选择**交互式变基**，在弹出的变基对话框中，**右键**你要修改的那一行提交 -> 选择 **“编辑提交 (Edit Commit)”**。只需要打开本地的代码文件，把刚才复制的修改粘贴进去，保存。

变基

变基 (Rebase) 用于将当前分支的提交重新应用到另一分支的顶端，从而保持线性历史。在 SourceTree 中右键点击当前分支，选择 `Rebase`，在弹出的对话框中选择目标分支（如 `master`），点击 `OK` 即可完成变基。变基后提交历史为线性，没有合并节点，历史更清晰。

```
git checkout feature-login
git rebase master
```



- **合并 (Merge)** = 把两条路汇合，产生一个新的交叉路口（合并提交 M）。
- **变基 (Rebase)** = 把其中一条路的起点拿起来，**转接到另一条路的最新位置**，让历史变成一条直线。

变基过程当中可能会产生冲突，要手动处理完冲突之后才能继续变基。

把当前分支变基到目标分支上，在 `feature` 分支上，执行 `git rebase main`，就是把 `feature` 变基到 `main` 上。

执行变基时，Git 并不会立即删除原来的节点，**原来的节点依然存在**，但没有任何分支指向它，在 SourceTree 或命令行里默认不显示，也就是变成了悬空节点。

交互式变基

交互式变基允许在“重演”提交的过程中，对提交进行修改、合并、删除或调整顺序。

与普通变基不同的事，交互式变基可以修改提交历史，对进行交互式变基的分支上的节点可以合并，修改，删除。

一旦分支被推送到了远程仓库，尤其合作分支，禁止对它进行变基，因为变基会改变提交的哈希值，导致其他人的本地历史与你强制推送的新历史“分叉”，造成灾难性的合并冲突。

贮藏

点击 `Stash` 按钮，将当前未提交的变更暂存起来，工作区恢复为干净状态，便于切换分支或拉取代码。

```
git stash          # 贮藏当前变更
git stash pop      # 恢复最近一次贮藏
```

贮藏 (Stash) 是**全局的**，不绑定某个特定分支。它存储在 `.git/refs/stash` 中，独立于分支之外。

但是应用贮藏时，当前分支的工作区必须是干净的（没有未提交的修改），否则 SourceTree 会阻止应用。

应用贮藏时可能出现冲突，Git 会将贮藏中的修改与当前分支的代码进行**三方合并**：

情况	结果
修改的代码在目标分支中没有变化	顺利应用
修改的代码在目标分支中被其他人改过	产生冲突 ，需要手动解决
修改的文件在目标分支中被删除	产生冲突

除了应用贮藏 (Apply Stash) 之外也可以**弹出贮藏** (Pop Stash)，两者都是将贮藏内容恢复到工作区，区别是弹出贮藏会**删除**贮藏列表里对应的项（贮藏从列表中移除）

Source里没有弹出贮藏选项，可通过应用贮藏后删除贮藏实现

可以在应用贮藏后恢复已删除但是贮藏过的文件

应用贮藏只恢复文件内容（工作区），不改变提交状态。恢复后的文件会处于“未暂存变更”状态，需要手动执行 `暂存` 和 `提交` 才能记录到版本库中。

丢弃

右键点击已修改的文件，选择 `Discard` 即可丢弃该文件的所有未提交变更，恢复到最近一次提交的状态。

丢弃就是**放弃当前工作区（未暂存文件）中所有的修改，将文件强制恢复到上一次提交（HEAD）时的状态**。这是一个**不可逆**的操作，丢弃后改动将彻底丢失。

```
git checkout -- <文件名>
```

丢弃会删除未提交的本地文件。丢弃操作只会影响**工作区（未暂存）**的文件，不会影响已暂存 (Stage) 或已提交 (Commit) 的内容。若需同时清空暂存区，需先执行“取消暂存”再丢弃。

丢弃的本质是**直接将工作区覆盖为 HEAD 指针指向的版本**，因此不涉及任何合并或冲突解决。在 SourceTree 中，右键点击文件 → 选择“丢弃”，即可快速撤销未提交的修改。

情况	结果
文件只存在于工作区（未暂存）	直接恢复为 HEAD 的旧版本
文件已被 <code>git add</code> （已暂存）	需先“取消暂存”，才能丢弃
文件在本地被新增但未 <code>git add</code>	可通过 <code>git clean</code> 删除
修改已提交 (Commit)	丢弃不生效，需用 <code>git reset</code> 或 <code>git revert</code>

移除

右键点击文件选择 **Remove** 将从版本库中删除文件并标记为已删除，需要提交后才生效。

```
git rm <文件名>
git commit -m "删除文件"
```

可以移除文件，某次提交，贮藏记录，**最危险的是“移除未跟踪文件”和“删除贮藏记录”**，这两者会**永久丢失数据**，

移除主要是为了删除关系和撤销操作

移除目标	“移除”的含义	底层命令
文件状态 → 已跟踪文件	从 Git 中删除， 但保留硬盘文件	<code>git rm --cached</code>
文件状态 → 未跟踪文件	从 Git 中删除， 且物理删除硬盘文件	<code>rm + git rm</code>
提交历史 → 某个提交	撤销该提交（Revert）	<code>git revert</code>
贮藏列表 → 某条记录	永久删除贮藏	<code>git stash drop</code>
标签列表 → 某个标签	删除本地标签	<code>git tag -d</code>

创建标签

右键点击某个提交节点，选择 **Tag**，输入标签名称即可为该提交打标签。

```
git tag <标签名>
git push origin <标签名> # 推送标签到远程
```

删除标签

```
git tag -d <标签名> # 删除本地标签
git push origin --delete <标签名> # 删除远程标签
```

五.常见

快捷键	功能	说明
Ctrl + Shift + H	显示/隐藏已忽略的文件	切换是否显示被 <code>.gitignore</code> 忽略的文件
Ctrl + Shift + B	显示/隐藏书签栏	快速切换左侧仓库列表的显隐

快捷键	功能	说明
Ctrl + Shift + F	搜索文件/提交	在提交历史中快速查找
Ctrl + 1	切换到“文件状态”视图	查看当前工作区和暂存区的改动
Ctrl + 2	切换到“提交历史”视图	查看当前分支的提交记录
Ctrl + 3	切换到“分支/标签”视图	管理本地和远程分支
Ctrl + 4	切换到“贮藏”视图	查看和管理贮藏记录
Ctrl + Enter	提交 (Commit)	在提交信息框里按下，快速提交
Ctrl + Shift + P	推送 (Push)	将本地提交推送到远程仓库
Ctrl + Shift + U	拉取 (Pull)	从远程仓库拉取并合并更新
Ctrl + Shift + R	刷新 (Refresh)	刷新当前仓库的状态
Ctrl + W	关闭当前仓库	关闭当前打开的仓库标签页
Ctrl + D	打开差异对比窗口	查看选中文件的具体改动内容
Esc	关闭弹出窗口	快速关闭各种对话框或菜单
Ctrl + F	在当前视图搜索	在提交历史或文件列表中搜索
F5	刷新当前视图	等同于 Ctrl + Shift + R
Enter	展开/折叠选中项	在提交历史或文件列表中展开详情

常见错误

- 不同分支文件互不可见，需执行 `Merge` 才能同步数据。
- 已存在提交的独立分支不可用 `Reset` 删改历史，否则会导致预期冲突消失。
- `Hard Reset` 只能用于合并后“被污染”的主分支；未合并的独立分支不可重置，否则代码会丢失。
- 新建空远程仓库时 `Pull` 不可用，必须先执行 `Push` 将本地历史推上去。

1. 状态机卡死

Git 内部有两种处理代码合并的底层机制。一种是 `rebase`（变基），另一种是 `am`（通常用来应用 `.patch` 补丁，即 `git am` 命令）。**执行操作时，第一步操作是执行动作B，但是此时Git是在动作A的中途，还没有完成。**此时会产生死锁，导致其他都做也无法运。通常是在交互式变基和应用补丁未完成的时候开始其他操作导致的。

解决办法：彻底关闭SourceTree，打开对应项目根目录，进入隐藏文件夹.git里删除rebase-rebase文件夹后重启。

所以在开始交互式变基或者应用补丁的时候就应该直接执行完毕再完成其他操作。

2. 冲突合并中断后残存状态（Merge/Rebase 卡壳）

执行合并（Merge）或变基（Rebase）遇到冲突时，若未完全解决冲突就直接关闭 SourceTree 或强行切换分支，会导致 Git 状态锁定。此时工作区“卡”在半成品状态，无法进行暂存、提交或拉取等常规操作。

text

```
git merge --abort      # 放弃当前合并
git rebase --abort     # 放弃当前变基
git status             # 查看是否仍处于合并/变基状态
```

解决办法：若在 SourceTree 中看到分支旁出现（Rebasing）或（Merging）的提示，说明状态未解除。应直接点击“操作”菜单，选择“解决冲突”→“放弃合并 / 放弃变基”。若按钮被锁死无法操作，则需借助命令行执行对应的 `--abort` 命令强制中断。

情况	结果
冲突只解决了一半就退出软件	Git 状态锁定，常规操作被拦截
点击 SourceTree 的“放弃变基”	清理残存状态，恢复正常分支
冲突文件处于暂存和非暂存的混合状态	需先“丢弃”或“取消暂存”清理文件，再尝试放弃

3. 误删未提交代码（意外丢弃）

在文件状态面板中右键点击“丢弃”或在终端执行 `git checkout` 时，若误操作将正在开发的未提交代码覆盖为 HEAD 版本，由于该动作不可逆，改动将永久丢失，无法通过 Git 本身找回。另外，`git clean -fd` 会直接删除所有未跟踪的新增文件。

text

```
# 没有直接的撤回命令！只能靠提前备份或 IDE 的历史记录
```

解决办法：**没有撤回的方法**。防范此类错误，养成在丢弃前将代码复制到记事本备份的习惯。对于未暂存的改动，常用的保底做法是：在执行丢弃操作前，先使用“贮藏”功能将修改保存到栈中，确认无误后再考虑丢弃。若删除的是未跟踪文件且已丢，只能依赖 IDE（如 VS Code）的“本地历史记录”功能或文件恢复工具尝试找回。

情况	结果
丢弃了正在编写的未提交代码	代码永久消失，Git 无法还原
执行 <code>git clean -fd</code> 删除新文件	文件直接从磁盘物理删除
丢弃前先执行 <code>git stash</code>	留存保底，错误后可重新应用

4. 推送被远程拒绝（非快进式冲突）

本地提交代码后尝试推送时，若远程仓库已有别人更新的代码，Git 会直接拒绝推送。这是因为本地的提交历史不包含远程的最新变更，此时强制推送（`git push -f`）将覆盖并删除别人的代码，极其危险。

text

```
git pull --rebase          # 拉取远程代码并以变基方式合并
git push origin 分支名    # 解决冲突并提交后，再次推送
```

解决办法：遇到拒绝时，**先拉取远程代码**。建议使用 `git pull --rebase`（拉取后放在本地最新提交后面），而不是直接合并，这样能保持历史记录呈线性。若拉取时产生冲突，需按步骤手动解决并提交，最后再执行推送。**严禁在公共分支上使用 `git push -f`**。

情况	结果
本地版本落后远程版本直接推送	远程拒绝，推送失败
在公共分支使用 <code>git push -f</code>	他人提交的历史记录被直接覆盖
执行 <code>git pull --rebase</code> 成功	本地历史线形干净，推送顺利